SketchBloom: Training-Free Vector Sketch Outpainting with Stroke-Level Style Control

ANONYMOUS AUTHOR(S)
SUBMISSION ID: 1591

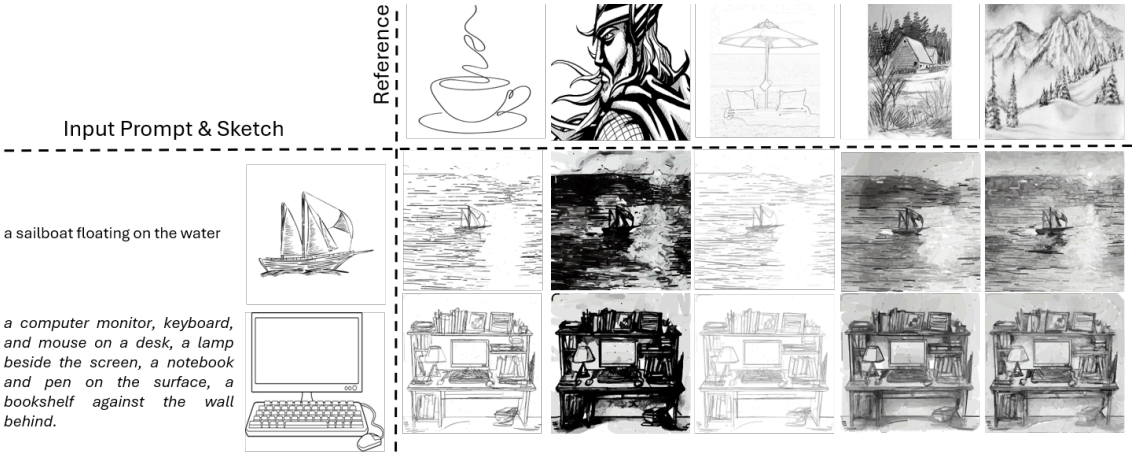


Fig. 1. **Style-guided vector sketch outpainting with SketchBloom.** The top row illustrates five diverse reference styles, ranging from clean linework to dense cross-hatching and loose painterly strokes. Each subsequent row pairs an input sketch and corresponding text prompt (left) with the outpainted SVG results conditioned on each respective style (columns). SketchBloom seamlessly extends the original sketch boundaries while faithfully transferring the distinct stroke vocabulary and visual identity of each reference.

While diffusion models excel at photorealistic image extension, outpainting sparse sketch and line art while preserving stroke-level structural coherence and stylistic identity remains an open challenge. Existing approaches either collapse contour structures into grayscale approximations or are restricted to generation-from-scratch paradigms. We present SketchBloom, a training-free framework for text-guided sketch outpainting that strictly maintains the sketch domain throughout the generation process. Given an input sketch, a text prompt describing the outpainted scene, and an optional reference style image, SketchBloom transfers the reference's visual identity (stroke width, color, and texture distribution) via an adaptive latent blending mechanism that provides explicit control over the structure-texture trade-off without any model retraining. To produce a resolution-independent Scalable Vector Graphic (SVG) composed of optimized Bézier curves, we introduce a structured differentiable rendering pipeline. This pipeline efficiently decouples structural contour strokes from free-form texture strokes, ultimately compositing them via a novel multiply-blend layering scheme. Experiments on SketchBloom-Bench, our newly curated benchmark of 1024 input-style combinations, demonstrate that SketchBloom consistently outperforms existing approaches across style fidelity and generation quality metrics.

CCS Concepts: • **Computing methodologies** → **Computer vision**; **Computer graphics**; • **Applied computing** → *Media arts*.

Additional Key Words and Phrases: Vector sketch synthesis, sketch outpainting, style transfer, differentiable rendering, stroke-based rendering, Bézier curves

This work is licensed under a Creative Commons Attribution 4.0 International License.
SA '26, Kuala Lumpur, Malaysia
© 2026 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-XXXX-X/2026/12
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Anonymous Author(s). 2026. SketchBloom: Training-Free Vector Sketch Outpainting with Stroke-Level Style Control. In *Proceedings of SIGGRAPH Asia 2026 (SA '26).* ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Sketching is a fundamental mode of visual expression, capturing structural intent and stylistic identity through sparse, deliberate strokes. While diffusion-based outpainting has achieved impressive results in the photorealistic domain [18, 20], extending a sketch beyond its original boundaries while preserving both structural coherence and stroke-level style remains an open challenge. Sketch outpainting requires strict adherence to the stroke domain; generated content must not only read as line art but also seamlessly reflect the original drawing's stylistic vocabulary.

Existing approaches struggle at this intersection. Photographic style transfer methods [5, 9] degrade on contour-dominant line art, often collapsing stroke structures into blurred grayscale approximations. Conversely, stroke-based rendering methods [22, 26] operate strictly in a generation-from-scratch paradigm, lacking the mechanisms to extend existing sketches or condition upon a reference style. Furthermore, even visually convincing raster-based methods [18, 30, 32] yield rigid pixel outputs that cannot be scaled or edited at the stroke level, making them unsuitable for integration into professional vector design pipelines. Bridging the gap between the semantic richness of diffusion models and the structural precision of vector graphics motivates our proposed approach.

To this end, we present **SketchBloom**[1], a novel training-free framework for style-guided sketch outpainting that outputs resolution-independent SVGs composed of optimized Bézier curves. To the best of our knowledge, SketchBloom is the first method to jointly address sketch outpainting, stroke-level style conditioning, and scalable vector output within a single training-free pipeline (Fig. 1). Given a sparse sketch, a text prompt, and an optional reference style image, SketchBloom comprises three main components. First, a ControlNet-based [32] outpainting stage semantically extends the sketch, followed by a texture-aware style transfer module that transfers the tonal and textural properties of the reference via adaptive latent blending over DDIM-inverted[20] feature sequences. We further introduce a hybrid stroke initialization strategy that combines contour-guided warping with DINO self-attention-guided placement [4]. This provides semantically grounded stroke coverage, drastically reducing the required optimization iterations to reach a faithful reconstruction. After that, using DiffVG [14], a structured differentiable optimization loop decouples the structure and texture stroke sets. Each set is optimized under a dedicated loss (*i.e.*, a patch-based multi-scale objective for structure and a global reconstruction loss for texture) before being composited via a novel multiply-blend layering scheme into the final SVG output.

To evaluate SketchBloom, we introduce **SketchBloom-Bench**, a comprehensive benchmark comprising 1,024 unique input-style combinations generated by pairing 64 outpainted content images across 47 sketch inputs with 16 reference styles spanning diverse rendering regimes. Extensive experiments and user study on this newly constructed dataset demonstrate that SketchBloom consistently outperforms existing vectorization methods across both style fidelity and generation quality metrics.

Our key contributions are as follows:

- We introduce SketchBloom, a novel training-free framework for style-guided sketch outpainting that produces scalable SVG outputs via differentiable Bézier curve optimization.
- We propose a hybrid stroke initialization strategy combining contour-guided warping with DINO self-attention-guided sampling, ensuring semantically grounded stroke coverage that substantially reduces optimization overhead.
- We design a patch-based multi-scale structure loss that stabilizes gradient signals on sparse contour targets by combining a fine-scale and a coarse-scale term over a uniform spatial grid, decoupled from a global texture objective.
- We present a multiply-blend vector fusion scheme that composites structure and texture stroke sets in Z-order, preserving contour dominance while retaining full-canvas textural richness.
- Extensive experiments on our new SketchBloom-Bench dataset, comprising 1,024 unique input-style combinations, demonstrate that SketchBloom consistently outperforms existing vectorization methods across both style fidelity and generation quality metrics.

[1]The name SketchBloom metaphorically captures the framework's core outpainting capability, where a sparse initial sketch organically blooms and extends beyond its original boundaries into a larger, stylistically consistent canvas.

2 Related Work

Vector Sketch Generation. Early vector sketch synthesis methods relied on gradient-free optimization over stroke parameters. DiffVG [14] enabled differentiable rasterization and end-to-end vector optimization through direct backpropagation, enabling CLIP-based perceptual supervision methods [6, 21, 22]. Subsequent approaches [13, 23, 26, 27] integrated diffusion priors and large language models for text-guided or autoregressive generation, while recent works have improved optimization efficiency and reconstruction fidelity through techniques like stroke pruning [10], splatting-based rendering [15], semantic region guidance [34], discrete token prediction with large language models [29], and direct raster-to-sketch prediction [2]. In contrast to prior methods, which focus primarily on generation-from-scratch or raster vectorization, SketchBloom explicitly targets sparse line-art outpainting combined with reference-guided style transfer. Our unified, training-free framework explicitly decouples structural and textural stroke objectives, a crucial trade-off for preserving the geometric precision necessary for contour-dominant sketches.

Sketch Style Transfer. Sketch style transfer differs fundamentally from photorealistic stylization, as semantics are encoded through sparse stroke geometry rather than dense appearance statistics. Consequently, standard Gram-optimization-based style transfer methods [8] and diffusion models [5, 9, 30] often collapse contour-dominant line art into blurred grayscale approximations. To address this, sketch-specific methods have improved stylization by utilizing stroke attention [28], transformer attention-guided differentiable optimization [24], reference-guided correspondence learning [31], multi-reference blending [35], and jointly controlled sketch abstraction and stylistic appearance [16]. Building upon these attention-guided philosophies, SketchBloom leverages DINO self-attention [4] for semantically informed stroke initialization, extending these concepts into the previously unexplored joint setting of sketch outpainting and style transfer. Our framework also adapts feature injection specifically to sketch-to-vector domain through an adaptive latent blending strategy and explicit stroke-level attribute extraction.

3 Proposed Method

3.1 Overview

Given a sparse sketch input I_{sketch}, a text prompt P, and a reference style image I_{ref}, SketchBloom synthesizes a stylistically faithful SVG extension of the original line art. When no style reference is provided, I_{ref} defaults to I_{sketch} itself, allowing the pipeline to derive stylistic cues directly from the input sketch. The pipeline proceeds through three sequential stages.

Stage 1: Raster Preprocessing. The input sketch is outpainted via a ControlNet-conditioned [32] diffusion model to produce a semantically extended content image I_{cnt}, from which structural contours are extracted as edge map I_{edge}. A StyleID-inspired [5] style transfer mechanism then produces the stylized raster I_{sty} that fuses the tonal, textural, and colorimetric properties of I_{ref} onto I_{cnt}, serving as the perceptual optimization target for the texture refinement stage (Sec. 3.2, Fig. 2).

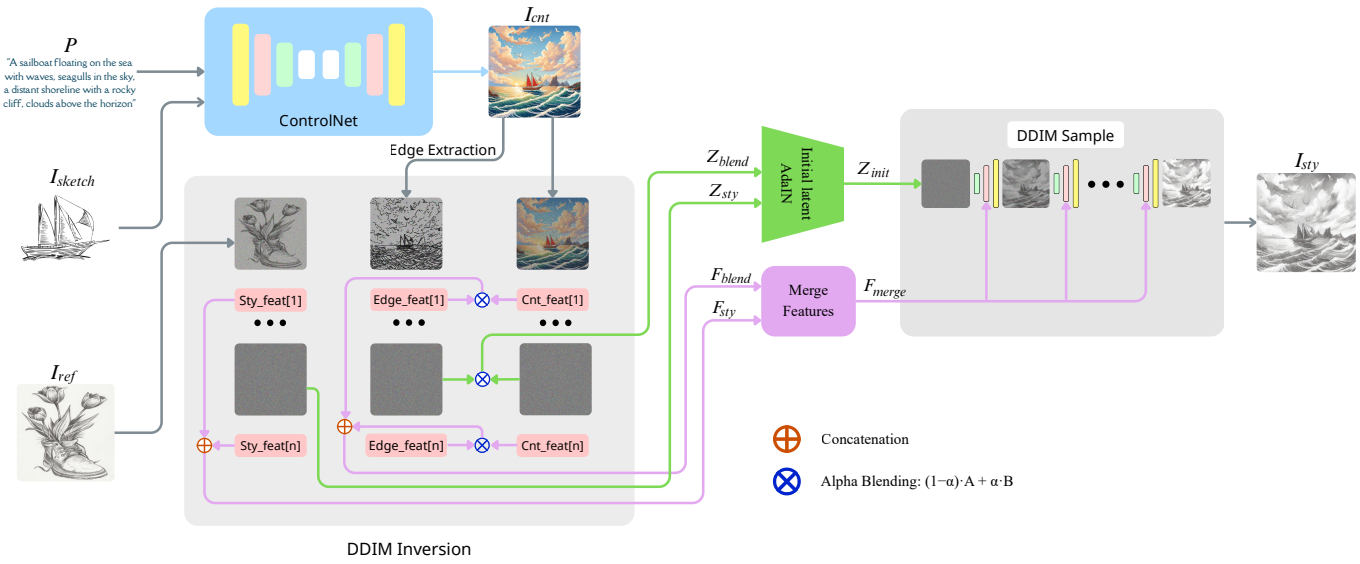


Fig. 2. **Raster preprocessing and style-guided outpainting pipeline of SketchBloom.** Given a sketch I_{sketch}, an optional reference style image I_{ref} (defaulting to I_{sketch} if not provided), and a text prompt P, a ControlNet first synthesizes a content image I_{cnt}. An edge map I_{edge} is extracted from I_{cnt}. All three images, I_{cnt}, I_{edge}, and I_{ref}, are independently processed via DDIM Inversion to extract per-timestep feature sequences and terminal latents. The resulting features are passed to two parallel branches: (1) an Initial Latent branch applying AdaIN to produce z_{init}, and (2) a Feature Assembly branch constructing $\mathcal{F}^{(i)}_{\text{merge}}$ by routing queries from content features and keys/values from style features. DDIM Sampling then denoises from z_{init} with $\mathcal{F}^{(i)}_{\text{merge}}$ injected at each timestep, producing the stylized output I_{sty}.

Stage 2: Differentiable Stroke Initialization and Optimization. A hybrid tiling strategy combines contour-guided and attention-guided placement to initialize N Bézier strokes $\{s_i\}_{i=1}^{N}$ over the canvas (Sec. 3.3, , Fig. 3). These strokes are refined through a structured optimization loop that separately aligns stroke geometry to I_{edge} and stroke appearance to I_{sty}, producing a structure stroke set $\mathcal{V}_{\text{struct}}$ and a texture stroke set $\mathcal{V}_{\text{tex}}$ (Sec. 3.4, Fig. 3).

Stage 3: Vector Fusion. $\mathcal{V}_{\text{struct}}$ and $\mathcal{V}_{\text{tex}}$ are composited into a unified scalable SVG via multiply-blend layering, where dark structural contours selectively suppress the underlying texture while light regions remain visually unchanged (Sec. 3.5, Fig. 3).

3.2 Raster Preprocessing

The raster preprocessing pipeline produces the stylized raster I_{sty} through two sequential steps: sketch outpainting and texture-aware style transfer.

Sketch Outpainting. The preprocessed sketch I_{sketch} is centrally composited onto an expanded canvas of size $r \cdot W \times r \cdot H$ and resized to 1024×1024 pixels. This expanded sketch serves as the control signal to ControlNet [32] paired with Stable Diffusion [18], conditioned on the text prompt P, running for 50 denoising steps with guidance scale 7.5 and conditioning scale 1.0. The resulting outpainted image I_{cnt} semantically extends the original drawing beyond its original boundaries. An edge detector is subsequently applied to I_{cnt} to obtain the contour map I_{edge}, which is used in downstream vector stages.

Texture-Aware Style Transfer. SketchBloom adopts a feature injection mechanism inspired by StyleID [5] feature injection mechanism to transfer the tonal and textural properties of I_{ref} into I_{cnt}. When I_{ref} is not provided, it defaults to I_{sketch}. All three images (*i.e.*, I_{cnt}, I_{edge}, and I_{ref}) are independently inverted via DDIM inversion [20], yielding terminal latents z_{cnt}, z_{edge}, z_{sty} and per-timestep self-attention feature sequences $\mathcal{F}^{(i)}_{\text{cnt}}$, $\mathcal{F}^{(i)}_{\text{edge}}$, $\mathcal{F}^{(i)}_{\text{sty}}$.

Adaptive Latent Blending. Rather than initializing generation directly from z_{cnt}, the content and edge latents are linearly interpolated as: $z_{\text{blend}} = (1-\alpha)\, z_{\text{edge}} + \alpha\, z_{\text{cnt}}$, where $\alpha \in [0,1]$ controls the structure-texture trade-off: $\alpha \to 0$ biases generation toward sharp contours derived from I_{edge}, while $\alpha \to 1$ retains richer tonal variation from I_{cnt} (Fig. 8). Therefore, we empirically set $\alpha = 0.5$ to balance trade-off between structural fidelity and stylistic richness. The same interpolation is applied to the corresponding feature sequences across all timesteps: $\mathcal{F}^{(i)}_{\text{blend}} = (1-\alpha)\mathcal{F}^{(i)}_{\text{edge}} + \alpha\mathcal{F}^{(i)}_{\text{cnt}}$.

Prior to denoising, AdaIN [11] aligns the channel-wise statistics of z_{blend} to those of z_{sty}: $z_{\text{init}} = \text{AdaIN}(z_{\text{blend}},\ z_{\text{sty}})$, ensuring the generation starting point already reflects the reference style distribution.

Feature Assembly and Injection. At each denoising timestep i, the blended feature sequence $\mathcal{F}^{(i)}_{\text{blend}}$ and the style feature sequence $\mathcal{F}^{(i)}_{\text{sty}}$ are assembled into a merged feature map $\mathcal{F}^{(i)}_{\text{merge}}$ according to the following routing strategy. The key K and value V tensors of each UNet self-attention layer are always sourced from $\mathcal{F}^{(i)}_{\text{sty}}$, transferring

the textural statistics of I_{ref}. The query Q transitions from style-sourced to blend-sourced after a predefined threshold timestep T_q, allowing early denoising steps to be globally guided by the reference style while later steps recover content-consistent spatial structure.

The self-attention output is further blended as:

$$O^{(i)} = (1 - w_i)\, O^{(i)}_{\text{blend}} + w_i\, O^{(i)}_{\text{sty}}, \tag{1}$$

where w_i follows a piecewise schedule: it remains constant at w_0 for $i \le T_w$, then decays via the quintic smoothstep

$$f(t) = 6t^5 - 15t^4 + 10t^3, \qquad t \in [0, 1], \tag{2}$$

as $w_i = w_0 \left(1 - f\left(\frac{i - T_w}{I - T_w}\right)\right)$ for $i > T_w$, where I is the total number of denoising steps. This allows style injection to remain strong during early denoising before progressively relaxing in the final steps to recover content structure. The assembled $\mathcal{F}^{(i)}_{\text{merge}}$ is injected into the UNet at each active timestep i during DDIM sampling from z_{init}, yielding the stylized raster I_{sty} that serves as the optimization target for all downstream vector stages.

3.3 Stroke Initialization via Hybrid Tiling

Given the content image I_{cnt} and its edge map I_{edge}, SketchBloom constructs an initial stroke set $\{s_i\}_{i=1}^{N}$ through a hybrid tiling strategy that combines contour-guided placement and attention-guided sampling. The stroke budget N is partitioned into three groups: a majority fraction $\rho \in [0, 1]$ is allocated to contour-guided strokes, of which two-thirds are placed directly along detected contours and the remaining third are generated by interpolating between adjacent pairs; the rest are placed via attention-guided sampling.

Stroke Vocabulary. Prior to hybrid tiling, a proxy stroke set is initialized directly from I_{ref} via SLIC superpixel segmentation [1]. Each segment yields one stroke: the two farthest convex-hull border points define the stroke endpoints (s_i, e_i), the midpoint $c_i = (s_i + e_i)/2$ serves as the Bézier control point, and the mean segment color gives stroke color $\mathbf{c}_i$. Stroke width w_i is estimated as the average perpendicular distance from each border point to the axis $\overrightarrow{s_i e_i}$:

$$w_i = \frac{1}{|\mathcal{B}_i|} \sum_{p \in \mathcal{B}_i} \frac{\left|\overrightarrow{s_i e_i} \times \overrightarrow{s_i p}\right|}{\|\overrightarrow{s_i e_i}\|}, \tag{3}$$

where $\mathcal{B}_i$ denotes the set of convex-hull border points of segment i. This proxy set is then optimized against I_{ref} via differentiable rendering to refine all attributes, yielding a stroke vocabulary that captures the visual character of the reference style. Without this step, placed strokes would carry no stylistic information, as their color, width, and geometry would be initialized randomly with $\mathbf{c}_i \sim \mathcal{U}(0, 1)^3$ and positions drawn from a random walk of step radius 0.05 scaled to canvas size.

Contour-Guided Placement. Edge contours are extracted from I_{edge} by thresholding and connected-component analysis, then weighted proportionally to their arc length for stroke distribution. For each placement position along a contour, a template stroke is retrieved from the vocabulary via spatial grid lookup: the canvas is partitioned into a $G \times G$ grid, and the nearest reference strokes to the query position are sampled as candidates.

The selected template's control points are scaled and warped to conform to the local contour geometry. Each control point is projected onto the template's principal axis $\mathbf{u}_k$ (the leading right singular vector of the mean-centered control points) to obtain a normalized arc-length parameter $t_j \in [0, 1]$. Let $\bar{p}_j = p_j - \bar{p}$ be the centered coordinate of control point j, and let $\mathbf{r}_j = \bar{p}_j - (\bar{p}_j \cdot \mathbf{u}_k)\, \mathbf{u}_k$ be its perpendicular residual. Denoting by $\mathbf{n}(t_j) = (-\tau_y, \tau_x)$ the unit contour normal at t_j, the warped control point is:

$$\tilde{p}_j \;=\; C_k(t_j) \;+\; \beta\, \operatorname{sgn}(\mathbf{r}_j \cdot \mathbf{n}(t_j))\, \|\mathbf{r}_j\|\, \mathbf{n}(t_j), \qquad \beta = 0.3, \tag{4}$$

which re-projects the perpendicular residual from the template's local frame onto the contour normal direction, preserving stroke shape while adapting to the local geometry of C_k. Additional strokes are interpolated between adjacent placed pairs using a quintic smoothstep schedule (Eq. 2) to ensure smooth attribute transitions; pairs whose dissimilarity in color, width, and position falls below $\tau = 0.3$ are skipped to avoid redundancy.

Attention-Guided Placement. For semantically salient regions not covered by detected contours, strokes are sampled from an attention probability map $\mathbf{A}$ derived from the [CLS] token self-attention of a pretrained DINO ViT [4]. Attention weights from the final transformer block are averaged across heads and upsampled to canvas resolution. The edge confidence map $\mathbf{E}$ is obtained analogously from raw contour values. Both maps are independently softened via a power transform with exponents $\gamma_{\mathbf{A}} = 0.5$ and $\gamma_{\mathbf{E}} = 0.7$ respectively, to suppress noise while preserving salient regions.

When the spatial prior is enabled, each map is linearly blended with a spatial density map $\mathbf{D}$ derived from the proxy stroke set extracted on I_{ref} (Sec. 3.3, "Stroke Vocabulary"):

$$\mathbf{A}' = \lambda\, \mathbf{D} + (1 - \lambda)\, \mathbf{A}, \qquad \mathbf{E}' = \lambda\, \mathbf{D} + (1 - \lambda)\, \mathbf{E}, \qquad \lambda = 0.5. \tag{5}$$

$\mathbf{D}$ encodes the *template* stroke density of the reference style and acts as a *consolidation prior* rather than a uniformity prior: regions that are densely covered in I_{ref} receive proportionally more sampled strokes, which preserves the global stroke-mass distribution of the reference. The mixing weight λ thus balances adherence to the reference layout ($\mathbf{D}$) against semantic salience ($\mathbf{A}$, $\mathbf{E}$). Each blended map is renormalized to a probability distribution before sampling. Each attention-guided stroke is positioned by sampling a canvas location from the blended map, centering the template at that location, and applying a random spatial offset for variety. The final pool $\{s_i\}_{i=1}^{N}$ is the union of contour-placed, interpolated, and attention-sampled strokes.

3.4 Stroke Optimization and Refinement

The initialized stroke set $\{s_i\}_{i=1}^{N}$ is refined through two parallel optimization steps (Fig. 3). Both steps inherit their stroke-level priors, including width w, color $\mathbf{c}$, and distribution $\mathbf{D}$, from the *stroke vocabulary* computed in Sec. 3.3 on the proxy stroke set extracted from I_{ref}. We emphasize that vocabulary extraction is performed *before* hybrid tiling and is not part of the canvas refinement loop, since optimizing canvas-placed strokes directly against I_{ref} would destroy their spatial layout. The two refinement passes below operate on the canvas stroke set $\{s_i\}_{i=1}^{N}$ produced by hybrid tiling.

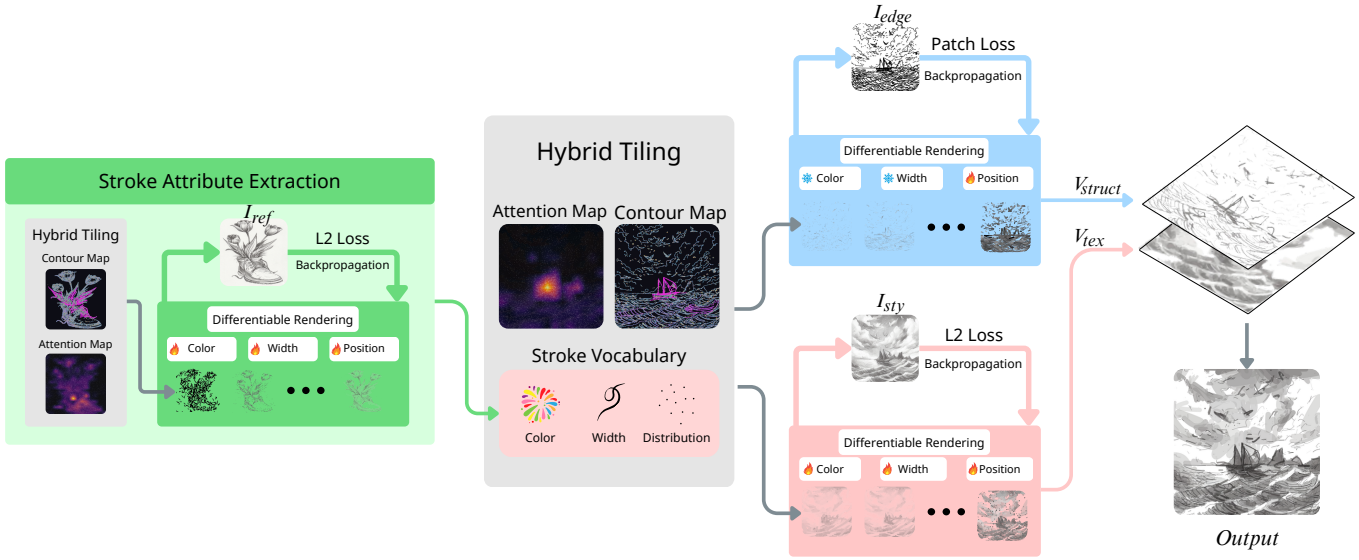


Fig. 3. **Stroke initialization, optimization, and vector fusion pipeline of SketchBloom.** A stroke vocabulary, comprising color, width, and spatial distribution, is first extracted by optimizing a proxy stroke set against I_{ref} via differentiable rendering. The learned attributes then initialize two parallel optimization passes: (1) a *structure pass* that fixes color and width while optimizing only stroke positions to align with the content edge map I_{edge}, producing the structure set $\mathcal{V}_{\text{struct}}$; and (2) a *texture pass* that jointly optimizes all attributes against the stylized raster I_{sty}, producing the texture set $\mathcal{V}_{\text{tex}}$. The two sets are composited as ordered layers, $\mathcal{V}_{\text{tex}}$ beneath and $\mathcal{V}_{\text{struct}}$ above under `multiply` blending, to yield the unified SVG output $\mathcal{V}^*$.

Step 1: Structure Optimization. With w and $\mathbf{c}$ held fixed, only geometric parameters $\{\mathbf{P}_i\}$ are optimized toward I_{edge}. Since I_{edge} exhibits highly uneven spatial density — dense along salient contours and empty elsewhere — a global loss averages over regions of vastly different complexity and produces inconsistent gradient signal across stroke locations. We therefore adopt a patch-based multi-scale objective over a $G \times G$ spatial grid:

$$\mathcal{L}_{\text{struct}} = \frac{1}{G^2} \sum_{i=1}^{G} \sum_{j=1}^{G} \Big(\mathcal{L}_{fine}(\hat{I}_{ij},\, I^e_{ij}) + \mathcal{L}_{coarse}\Big(\downarrow_r \hat{I}_{ij},\, \downarrow_r I^e_{ij}\Big)\Big), \quad (6)$$

where $\hat{I}_{ij}$ and I^e_{ij} are the rendered and target crops at patch (i, j), and $\downarrow_r$ denotes average pooling with factor $r = 4$. The fine-scale $\mathcal{L}_{fine}$ is robust to the sharp binary transitions characteristic of edge maps: its constant-magnitude gradient $\nabla_{\hat{x}} \mathcal{L}_{fine} = \text{sign}(\hat{x} - x)$ avoids the grey hedging that the coarse-scale $\mathcal{L}_{coarse}$ induces when stroke placement is uncertain. $\mathcal{L}_{coarse}$ operates on pooled patches where smoothed transitions eliminate this hedging effect, anchoring the low-frequency spatial distribution of stroke mass without competing with the fine-scale objective. The result is the structure stroke set $\mathcal{V}_{\text{struct}}$, whose geometry conforms to I_{edge} while retaining the stroke vocabulary of I_{ref}.

Step 2: Texture Optimization. A separate stroke set, re-initialized via the hybrid tiling strategy of Sec. 3.3, is optimized with all parameters $\{\mathbf{P}_i, w_i, \mathbf{c}_i\}$ free against I_{sty}:

$$\mathcal{L}_{\text{tex}} = \|\mathcal{R}(\{s_i\}) - I_{\text{sty}}\|_2^2. \quad (7)$$

Since I_{sty} distributes tonal and textural variation across the full canvas, the global $\mathcal{L}_{coarse}$ objective provides stable full-canvas convergence without over-fitting to patch-level artifacts, yielding the texture stroke set $\mathcal{V}_{\text{tex}}$.

3.5 Layer-wise Multiply Fusion

The unified output is defined as the ordered concatenation $\mathcal{V}^* = \mathcal{V}_{\text{tex}} \parallel \mathcal{V}_{\text{struct}}$, with $\mathcal{V}_{\text{tex}}$ on the lower layer and $\mathcal{V}_{\text{struct}}$ on the upper layer; the two stroke sets are embedded as independent SVG groups. The SVG is exported as two layers: $\mathcal{V}_{\text{tex}}$ is rendered under the `normal` compositing mode, while $\mathcal{V}_{\text{struct}}$ uses `mix-blend-mode: multiply`, so each structure stroke colour c^s_{xy} blends with the underlying texture colour c^t_{xy} as: $c^*_{xy} = c^s_{xy} \odot c^t_{xy}$, where $\odot$ denotes element-wise multiplication in $[0, 1]^3$. Dark structural contours thereby suppress the underlying texture locally, while regions where $c^s_{xy} \approx \mathbf{1}$ remain visually unchanged.

4 Experiments

4.1 Benchmark Dataset

To evaluate style-guided sketch outpainting comprehensively, we introduce the SketchBloom-Bench dataset, comprising 1,024 unique input-style combinations. We constructed this benchmark starting from 47 diverse base sketches, from common daily objects to organic subjects including animals and human figures, that naturally vary in stroke density and spatial complexity. Each input sketch was outpainted using one of two prompt complexity levels: *complex* prompts yielding 43 instances of full multi-object scenes, and *moderate* prompts yielding 21 instances of simpler compositions. This initial generation process produced a total of 64 distinct outpainted

Table 1. **Quantitative comparison of sketch vectorization performance on outpainted contents.** All evaluated methods are provided with the same outpainted content image (I_{cnt}) as input. Structural fidelity (Edge F1) is measured against the extracted edge map I_{edge}, while other metrics are evaluated directly against I_{cnt}. Best results are highlighted in **bold**.

Method	Aesthetic [19] ↑	DINO [4] ↑	DreamSim [7] ↓	Edge F1 ↑
CLIPasso [22]	4.1690	0.2064	0.7256	0.5262
CLIPascene [21]	4.3904	0.2022	0.7024	0.5797
SwiftSketch [2]	3.9784	0.1566	0.8305	0.3139
ControlSketch [2]	4.1493	0.1470	0.7960	0.3299
DiffSketcher [26] (adapted)	4.8020	0.3760	0.5351	0.8325
SketchBloom (Ours)	**5.2186**	**0.5053**	**0.4983**	**0.8866**

content images. Finally, each of the 64 content images was paired with 16 different reference styles, which are strategically organized into environmental *Scene* styles and isolated *Object* styles. These references cover a broad spectrum of visual complexities, ensuring the framework is rigorously evaluated across diverse rendering regimes. Experiments were conducted on all 1,024 input-style combinations.

4.2 Evaluation Metrics

We evaluate SketchBloom across four core dimensions: perceptual quality, style fidelity, structural fidelity, and geometric precision. To assess **perceptual quality**, we use the *Aesthetic Score* [19], a no-reference proxy for human-calibrated visual appeal, alongside *LPIPS* [33] and *DreamSim* [7], which measure deep feature distances and holistic visual similarity, respectively. Additionally, *Laplacian Variance*[17] serves as a no-reference sharpness estimator, where higher values indicate crisper, more committed stroke edges in the output.

For **style fidelity**, *Gram Similarity* [8] computes the feature-space distance against the reference style, while the *DINO Score* [4] provides a semantically grounded measure of visual alignment against the stylized raster. **Structural fidelity** is evaluated using *SSIM* [25], which accounts for luminance, contrast, and local structure, alongside *Mean Absolute Error (MAE)* and *Harmonic Mean of Precision and Recall (Edge F1)*, which calculate pixel-wise differences and boundary precision against specified ground-truth reference maps. Finally, **geometric precision** is measured using *Hausdorff Distance* [12] and *Chamfer Distance* [3]. Both metrics operate on point sets sampled from skeletonized stroke paths, effectively capturing large localized deviations and the overall geometric alignment of the generated strokes.

4.3 Comparison against State-of-the-Arts

No existing method simultaneously addresses sketch outpainting and explicit stroke-level style control. Therefore, our evaluation focuses strictly on **sketch vectorization quality from a given outpainted raster image**. We compare SketchBloom against five baselines, including CLIPasso [22], CLIPascene [21], SwiftSketch [2], ControlSketch [2], and an adapted DiffSketcher [26], using the same outpainted content image I_{cnt} as input for all. DiffSketcher [26] is adapted by replacing its original text-conditioned optimization target with I_{cnt} directly, enabling image-guided vector synthesis in place of its text-to-SVG pipeline.

Qualitative results in Fig. 4 demonstrate SketchBloom's superior visual performance. ControlSketch and SwiftSketch produce sparse, disconnected strokes that fail to capture complex scene structures, whereas CLIPasso generates noisy, irregular stroke placements that obscure underlying geometry. While CLIPascene captures coarse layouts, it over-abstracts fine details into flat silhouettes, and the adapted DiffSketcher produces softer strokes that overly smooth fine boundaries and textures. In contrast, SketchBloom consistently generates clean, well-structured stroke layouts that faithfully preserve object contours, perspective lines, and scene depth, seamlessly extending the original sketch boundaries while maintaining a coherent aesthetic style.

Table 2. **Structural fidelity comparison of stroke initialization strategies**, evaluated on the structure stroke set $\mathcal{V}_{\text{struct}}$ *in isolation* (*i.e.*, before multiply-blend composition with $\mathcal{V}_{\text{tex}}$). All metrics are computed against the contour map I_{edge} as ground truth. Best results are highlighted in **bold**.

Init.	MAE ↓	SSIM [25] ↑	Chamfer [3] ↓	Edge F1 ↑
Random	0.4384	0.1120	0.0006	0.7731
Hybrid	**0.2041**	**0.8068**	**0.0003**	**0.8841**

Table 3. **Texture and perceptual quality comparison of stroke initialization strategies**, evaluated on the texture stroke set $\mathcal{V}_{\text{tex}}$ *in isolation* (*i.e.*, before multiply-blend composition with $\mathcal{V}_{\text{struct}}$). Gram is measured against I_{ref}; Edge F1 is computed against I_{edge}; other remaining metrics are computed against I_{sty}. Best results are highlighted in **bold**.

Init.	Gram [8] ↓	SSIM [25] ↑	DreamSim [7] ↓	Edge F1 ↑	DINO [4] ↑	LPIPS [33] ↓
Random	1.99×10^{-4}	0.6718	0.2459	0.7975	0.6378	0.4034
Hybrid	$\mathbf{1.82 \times 10^{-4}}$	**0.7375**	**0.1744**	**0.8067**	**0.7328**	**0.3539**

Quantitative evaluations confirm that SketchBloom significantly outperforms all baseline methods across every metric (Tab. 1). SketchBloom achieves the highest visual appeal with an Aesthetic score of 5.2186 and the strongest semantic alignment with a DINO score of 0.5053. Additionally, it maintains the highest structural fidelity with an Edge F1 score of 0.8866, while recording the lowest overall perceptual distance with a DreamSim score of 0.4983. These results quantitatively validate that SketchBloom successfully geometrically accurate and well-ordered stroke layouts, seamlessly blending structural fidelity with a cohesive visual style.

4.4 Ablation Study

Effect of Stroke Initialization Strategy. We compare two initialization strategies to validate the hybrid tiling design (Sec. 3.3). Random strategy samples control points uniformly at random over the canvas, whereas our hybrid strategy combines contour-guided placement with DINO self-attention-guided sampling. Tab. 2 and Tab. 3 demonstrate that hybrid strategy achieves the superior performance across all metrics. Hybrid initialization provides a semantically grounded starting point that converges to well-placed strokes significantly earlier than the persistently misaligned random initialization, substantially reducing the burden on the optimizer (Fig. 5).

Effect of Layer-wise Multiply Fusion. We validate the vector fusion strategy (Sec. 3.5) by comparing three variants: a structure-only output using exclusively $\mathcal{V}_{\text{struct}}$ like those in Tab. 2, a texture-only output using exclusively $\mathcal{V}_{\text{tex}}$ like those in Tab. 3, and our proposed fusion approach that composites both into a two-layer SVG where structure strokes multiply-blend over texture strokes.

Tab. 4 shows that while the texture-only variant achieves the best Gram similarity by optimizing solely against the stylized raster,

Table 4. **Quantitative comparison of vector fusion strategies**. Gram is measured against I_{ref}; DINO and DreamSim are computed against I_{sty}; Edge F1 is computed against I_{edge}. Best results are highlighted in **bold**.

Variant	Gram [8] ↓	DINO [4] ↑	DreamSim [7] ↓	Edge F1 ↑	Aesthetic [19] ↑
Texture only	$\mathbf{1.82 \times 10^{-4}}$	0.7328	0.1744	0.8067	5.0715
Structure only	2.76×10^{-4}	0.5800	0.2918	**0.8841**	4.8718
Fusion	2.19×10^{-4}	**0.7372**	**0.1712**	0.8629	**5.3507**

Table 5. **Quantitative comparison of reconstruction loss functions**. All metrics are computed against I_{edge}. Best results are highlighted in **bold**.

Loss	LPIPS [33] ↓	SSIM [25] ↑	Hausdorff [12] ↓	Laplacian [17] ↑
L_1 Loss	0.2106	0.7872	23.3068	18181.7089
L_2 Loss	**0.1860**	**0.8284**	23.0437	16576.9966
Patch Loss	0.1962	0.8068	**22.6493**	**18852.2313**

it suffers a significant drop in structural coherence. Conversely, the structure-only variant maintains high geometric precision, but lacks stylistic tonal richness. Our layer-wise fusion strategy effectively balances geometric precision and stylistic expressiveness, securing the highest Aesthetic, DINO, and DreamSim scores while maintaining a competitive Gram similarity. This complementary composition consistently delivers sharper object boundaries than texture-only results while retaining the crucial tonal richness absent in structure-only sketches (Fig. 7).

Effect of Training Loss Function. We investigate how the per-pixel reconstruction loss affects stroke quality by comparing L_1, L_2, and our patch-based losses (Sec. 3.4). Tab. 5 reveals that while L_1 loss misses fine details and L_2 loss introduces grey hedging, our proposed patch-based multi-scale loss successfully mitigates both issues. By combining fine and coarse-scale objectives, it achieves an optimal balance of structural metrics and highest Laplacian and Hausdorff scores to ensure crisp, committed strokes (Fig. 6).

4.5 User Study

To complement the automatic evaluation, we conducted a perceptual user study comparing SketchBloom against the two strongest baselines in terms of vectorization quality: CLIPascene [21] and DiffSketcher adapted [26].

The study involved 33 participants (aged between 16 and 40). We designed the study as a series of pairwise comparisons where participants evaluated two anonymized output sketches generated from the same content image by SketchBloom and another method. These image pairs were drawn from a diverse subset of the SketchBloom-Bench dataset to ensure broad coverage of subject categories and prompt complexities.

For each randomized pair, participants were asked to indicate their preference based on three independent criteria. **Perceptual Quality** evaluates how visually appealing, clean, and readable the generated sketch appeared. **Content Preservation** measures how accurately the sketch retained the main objects and scene layout from the original input. **Structural Coherence** assesss whether the object boundaries, contours, and overall spatial layout were clearly recognizable and logically constructed.

Results on Tab. 6 demonstrate a strong human preference for SketchBloom across all criteria. When compared to CLIPascene, SketchBloom achieved preference rates of 97.2% for perceptual quality, 96.8% for content preservation, and 96.5% for structural coherence, primarily because CLIPascene tends to over-abstract fine details into flat silhouettes. Against the structurally stronger adapted DiffSketcher, SketchBloom still maintained a clear advantage, securing preference rates of 87.4%, 84.9%, and 83.5% across the respective criteria. These findings confirm that SketchBloom's unified optimization pipeline yields sketches with distinctly superior perceptual clarity and boundary fidelity.

Table 6. **Human preference rates from user study.** Each cell indicates the percentage of pairwise comparisons in which SketchBloom was chosen over the corresponding baseline method. Higher percentages denote a stronger human preference for SketchBloom. PQ, CP, and SC denotes Perceptual Quality, Content Preservation, and Structural Coherence.

Method	PQ ↑	CP ↑	SC ↑
CLIPascene [21]	97.2%	96.8%	96.5%
DiffSketcher adapted [26]	87.4%	84.9%	83.5%

4.6 Failure Cases

SketchBloom exhibits two primary failure modes related to its handling of complex textures and heterogeneous stroke attributes. The first issue, *texture dominance*, occurs when a reference style features dense, high-contrast stroke patterning, such as richly shaded ink-wash illustrations. Because the texture stroke set operates without structural constraints, it can deposit strokes across semantic boundaries and visually overwhelm the structural contours during the multiply-blend fusion. This effect is most pronounced when the reference style has a significantly higher stroke density than the input sketch, resulting in obscured object silhouettes where the primary subjects become difficult to distinguish from the background. The second issue, *stroke irregularity*, arises during the structure optimization pass because stroke width and color are intentionally frozen to values extracted from the reference style's vocabulary. While this preserves stylistic identity, it introduces brittleness when the vocabulary contains highly heterogeneous stroke widths. In such cases, the optimizer may place thick, heavy strokes directly alongside thin contour lines, creating abrupt width discontinuities that fracture the perceived silhouette of the object and degrade the overall structural legibility of the output sketch.

5 Conclusion

This paper presented SketchBloom, a novel training-free framework for style-guided sketch outpainting that generates resolution-independent SVG through differentiable Bézier curve optimization. By unifying ControlNet-driven semantic outpainting, adaptive latent blending for stroke-level style transfer, and a structured vector optimization loop, SketchBloom successfully addresses a complex joint task unmatched by any existing individual method. Comprehensive evaluations on the newly curated SketchBloom-Bench dataset demonstrate that our approach consistently outperforms adapted vectorization baselines in both perceptual quality and structural fidelity, with ablation studies validating the efficacy of each core pipeline component. Future work will address texture dominance and stroke irregularity issues caused by complex styles through content-aware stroke budgets, soft attribute regularization.

References

[1] Radhakrishna Achanta, Appu Shaji, Kevin Smith, Aurélien Lucchi, Pascal Fua, and Sabine Süsstrunk. 2012. SLIC Superpixels Compared to State-of-the-Art Superpixel Methods. *IEEE Trans. Pattern Anal. Mach. Intell.* 34, 11 (2012), 2274–2282. doi:10.1109/TPAMI.2012.120

[2] Ellie Arar, Yarden Frenkel, Daniel Cohen-Or, Ariel Shamir, and Yael Vinker. 2025. SwiftSketch: A Diffusion Model for Image-to-Vector Sketch Generation. In *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference, SIGGRAPH Conference Papers 2025, Vancouver, BC, Canada, August 10-14, 2025*, Ginger Alford, Hao (Richard) Zhang, and Adriana Schulz (Eds.). ACM, 82:1–82:12. doi:10.1145/3721238.3730612

[3] Harry G. Barrow, Jay M. Tenenbaum, Robert C. Bolles, and Helen C. Wolf. 1977. Parametric Correspondence and Chamfer Matching: Two New Techniques for Image Matching. In *International Joint Conference on Artificial Intelligence*.

[4] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. 2021. Emerging Properties in Self-Supervised Vision Transformers. In *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*. IEEE, 9630–9640. doi:10.1109/ICCV48922.2021.00951

[5] Jiwoo Chung, Sangeek Hyun, and Jae-Pil Heo. 2024. Style Injection in Diffusion: A Training-Free Approach for Adapting Large-Scale Diffusion Models for Style Transfer. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*. IEEE, 8795–8805. doi:10.1109/CVPR52733.2024.00840

[6] Kevin Frans, Lisa B. Soros, and Olaf Witkowski. 2022. CLIPDraw: Exploring Text-to-Drawing Synthesis through Language-Image Encoders. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/21f76686538a5f06dc431efea5f475f5-Abstract-Conference.html

[7] Stephanie Fu, Netanel Tamir, Shobhita Sundaram, Lucy Chai, Richard Zhang, Tali Dekel, and Phillip Isola. 2023. DreamSim: Learning New Dimensions of Human Visual Similarity using Synthetic Data. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/9f09f316a3eaf59d9ced5ffaefe97e0f-Abstract-Conference.html

[8] Leon A. Gatys, Alexander S. Ecker, and Matthias Bethge. 2016. Image Style Transfer Using Convolutional Neural Networks. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2414–2423. doi:10.1109/CVPR.2016.265

[9] Amir Hertz, Andrey Voynov, Shlomi Fruchter, and Daniel Cohen-Or. 2024. Style Aligned Image Generation via Shared Attention. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*. IEEE, 4775–4785. doi:10.1109/CVPR52733.2024.00457

[10] Or Hirschorn, Amir Jevnisek, and Shai Avidan. 2024. Optimize & Reduce: A Top-Down Approach for Image Vectorization. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (Eds.). AAAI Press, 2148–2156. doi:10.1609/AAAI.V38I3.27987

[11] Xun Huang and Serge J. Belongie. 2017. Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization. In *IEEE International Conference on Computer Vision, ICCV 2017, Venice, Italy, October 22-29, 2017*. IEEE Computer Society, 1510–1519. doi:10.1109/ICCV.2017.167

[12] Daniel P. Huttenlocher, Gregory A. Klanderman, and William Rucklidge. 1993. Comparing Images Using the Hausdorff Distance. *IEEE Trans. Pattern Anal. Mach. Intell.* 15, 9 (1993), 850–863. doi:10.1109/34.232073

[13] Ajay Jain, Amber Xie, and Pieter Abbeel. 2023. VectorFusion: Text-to-SVG by Abstracting Pixel-Based Diffusion Models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*. IEEE, 1911–1920. doi:10.1109/CVPR52729.2023.00190

[14] Tzu-Mao Li, Michal Lukác, Michaël Gharbi, and Jonathan Ragan-Kelley. 2020. Differentiable vector graphics rasterization for editing and learning. *ACM Trans. Graph.* 39, 6 (2020), 193:1–193:15. doi:10.1145/3414685.3417871

[15] Xi Liu, Chaoyi Zhou, Nanxuan Zhao, and Siyu Huang. 2025. Bézier Splatting for Fast and Differentiable Vector Graphics Rendering. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=bTclOYRfYJ

[16] Min Lu, Yuanfeng He, Anthony Chen, Jia He, Puwei Wang, Daniel Cohen-Or, and Hui Huang. 2026. Abstraction in Style. https://api.semanticscholar.org/CorpusID:286979050

[17] Said Pertuz, Domenec Puig, and Miguel Ángel García. 2013. Analysis of focus measure operators for shape-from-focus. *Pattern Recognit.* 46 (2013), 1415–1432. https://api.semanticscholar.org/CorpusID:17364877

[18] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-Resolution Image Synthesis with Latent Diffusion Models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 10674–10685. doi:10.1109/CVPR52688.2022.01042

[19] Christoph Schuhmann. 2022. Improved Aesthetic Predictor. https://github.com/christophschuhmann/improved-aesthetic-predictor. Accessed: 2026-04-07.

[20] Jiaming Song, Chenlin Meng, and Stefano Ermon. 2021. Denoising Diffusion Implicit Models. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. https://openreview.net/forum?id=St1giarCHLP

[21] Yael Vinker, Yuval Alaluf, Daniel Cohen-Or, and Ariel Shamir. 2023. CLIPascene: Scene Sketching with Different Types and Levels of Abstraction. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. IEEE, 4123–4133. doi:10.1109/ICCV51070.2023.00383

[22] Yael Vinker, Ehsan Pajouheshgar, Jessica Y. Bo, Roman Christian Bachmann, Amit Haim Bermano, Daniel Cohen-Or, Amir Zamir, and Ariel Shamir. 2022. CLIPasso: semantically-aware object sketching. *ACM Trans. Graph.* 41, 4 (2022), 86:1–86:11. doi:10.1145/3528223.3530068

[23] Yael Vinker, Tamar Rott Shaham, Kristine Zheng, Alex Zhao, Judith E. Fan, and Antonio Torralba. 2025. SketchAgent: Language-Driven Sequential Sketch Generation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 23355–23368.

[24] Kai Wang, Fei Yang, and Joost van de Weijer. 2022. Attention Distillation: self-supervised vision transformer students need more guidance. In *33rd British Machine Vision Conference 2022, BMVC 2022, London, UK, November 21-24, 2022*. BMVA Press, 666. https://bmvc2022.mpi-inf.mpg.de/666/

[25] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE Trans. Image Process.* 13, 4 (2004), 600–612. doi:10.1109/TIP.2003.819861

[26] Ximing Xing, Chuang Wang, Haitao Zhou, Jing Zhang, Qian Yu, and Dong Xu. 2023. DiffSketcher: Text Guided Vector Sketch Synthesis through Latent Diffusion Models. (2023). http://papers.nips.cc/paper_files/paper/2023/hash/333e67fc4728f147d31608db3ca78e09-Abstract-Conference.html

[27] Ximing Xing, Haitao Zhou, Chuang Wang, Jing Zhang, Dong Xu, and Qian Yu. 2024. SVGDreamer: Text Guided SVG Generation with Diffusion Model. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*. IEEE, 4546–4555. doi:10.1109/CVPR52733.2024.00435

[28] Rui Yang, Huining Li, Yiyi Long, Xiaojun Wu, and Shengfeng He. 2025. Stroke2Sketch: Harnessing Stroke Attributes for Training-Free Sketch Generation. *CoRR* abs/2510.16319 (2025). arXiv:2510.16319 doi:10.48550/ARXIV.2510.16319

[29] Yiying Yang, Wei Cheng, Sijin Chen, Xianfang Zeng, Jiaxu Zhang, Liao Wang, Gang Yu, Xingjun Ma, and Yu-Gang Jiang. 2025. OmniSVG: A Unified Scalable Vector Graphics Generation Model. *CoRR* abs/2504.06263 (2025). arXiv:2504.06263 doi:10.48550/ARXIV.2504.06263

[30] Hu Ye, Jun Zhang, Sibo Liu, Xiao Han, and Wei Yang. 2023. IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models. *CoRR* abs/2308.06721 (2023). arXiv:2308.06721 doi:10.48550/ARXIV.2308.06721

[31] Kwan Yun, Youngseo Kim, Kwanggyoon Seo, Chang Wook Seo, and Junyong Noh. 2024. Representative Feature Extraction During Diffusion Process for Sketch Extraction with One Example. *CoRR* abs/2401.04362 (2024). arXiv:2401.04362 doi:10.48550/ARXIV.2401.04362

[32] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. 2023. Adding Conditional Control to Text-to-Image Diffusion Models. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. IEEE, 3813–3824. doi:10.1109/ICCV51070.2023.00355

[33] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. 2018. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 586–595. doi:10.1109/CVPR.2018.00068

[34] Hengyu Zhou, Hui Zhang, and Bin Wang. 2024. Segmentation-Guided Layer-Wise Image Vectorization with Gradient Fills. In *Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part X (Lecture Notes in Computer Science)*, Ales Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (Eds.). Springer, 165–180. doi:10.1007/978-3-031-72684-2_10

[35] Jin Zhou, Yi Zhou, Hongliang Yang, Pengfei Xu, and Hui Huang. 2026. StrokeFusion: Vector Sketch Generation via Joint Stroke-UDF Encoding and Latent Sequence Diffusion. In *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20-27, 2026*, Sven Koenig, Chad Jenkins, and Matthew E. Taylor (Eds.). AAAI Press, 13656–13664. doi:10.1609/AAAI.V40I16.38372

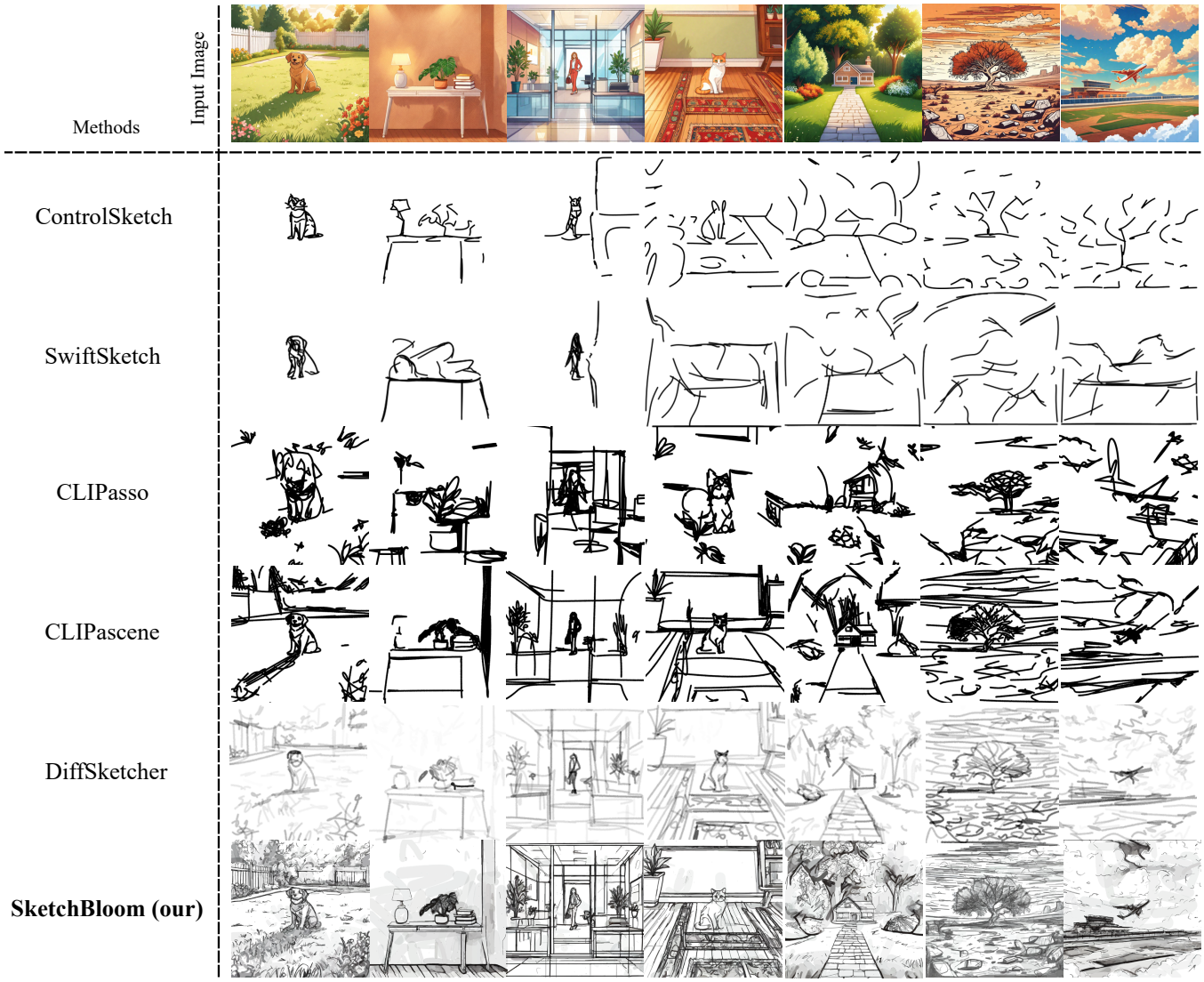


Fig. 4. **Qualitative comparison on sketch vectorization from outpainted content I_{cnt} (top row)**. All methods receive the same input image; the original object sketches and prompts are omitted for visualization clarity. ControlSketch [2] and SwiftSketch [2] (first two rows) produce sparse, highly abstract strokes that fail to recover scene structure in complex scenes. CLIPasso [22] (third row) generates denser but spatially disordered strokes with limited structural coherence. CLIPascene [21] (fourth row) captures coarse layout through bold, high-contrast strokes but over-abstracts fine spatial detail. DiffSketcher [26] (fifth row) produces lighter, more distributed strokes with softer boundaries yet under-represents local texture. Our SketchBloom (bottom row) produces spatially grounded outputs that more faithfully preserve object boundaries, perspective, and scene depth present in I_{cnt}.

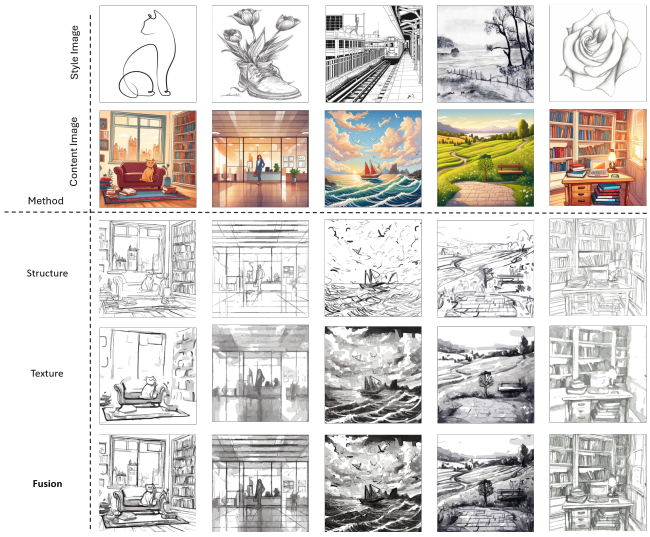


Fig. 7. **Qualitative comparison of the three fusion variants across five style-content pairs.** *Structure only* (first row) preserves edge geometry but lacks tonal variation and stylistic texture. *Texture only* (second row) faithfully captures the shading and stroke character of the reference style, yet produces spatially incoherent layouts with poorly defined contours. *Fusion* (third row) combines the geometric precision of structure strokes with the stylistic expressiveness of texture strokes, yielding outputs that are both structurally faithful and visually consistent with the target style.

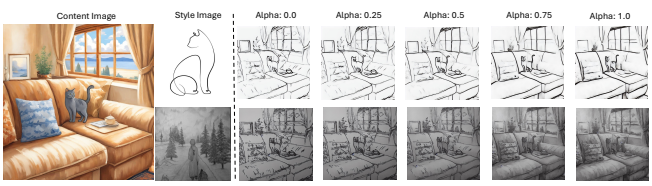


Fig. 8. **Effect of the adaptive latent blending parameter α on the style-transferred output I_{sty}.** The leftmost column shows the shared content image I_{cnt}; the second column shows the two reference style sketches used in each row. *Top row:* a sparse line-art reference. At $\alpha = 0$, contours are sharpest and most faithful to the structural boundaries of I_{cnt}; increasing α gradually softens edges and suppresses fine linework. *Bottom row:* a tonal, shaded reference. Low α values produce outputs with strong edge contrast but limited tonal depth; higher values admit progressively richer shading at the expense of contour definition. In both styles, $\alpha = 0.5$ achieves a visually balanced trade-off between structural fidelity and stylistic richness.



Fig. 5. **Optimization trajectory of random (odd rows) and hybrid (even rows) initialization at different iterations, evaluated on two sketches: a structured indoor scene (top two rows) and a landscape scene (bottom two rows).** For the indoor scene, random initialization produces strokes that drift off structural boundaries and converge slowly, whereas the hybrid strategy achieves well-aligned, semantically grounded coverage significantly earlier. The landscape scene further highlights this contrast: random initialization yields incoherent, off-contour stroke clusters that remain poorly structured even at later iterations, while hybrid initialization converges to a stable, boundary-aligned sketch well before the optimization concludes. Across both scenes, the results demonstrate that a semantically grounded starting point both accelerates convergence and improves final output quality under the same optimization budget.

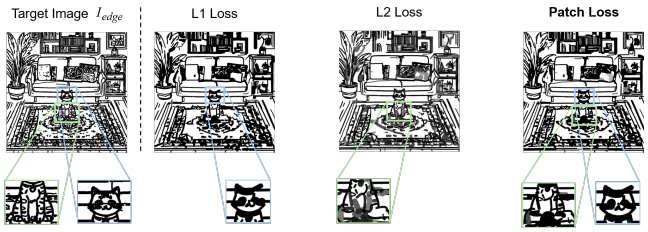


Fig. 6. **Qualitative comparison of L_1 loss (top), L_2 loss (middle), and patch loss (bottom) across three content images.** L_1 loss tends to produce globally consistent outlines but fails to resolve fine-grained local detail, most visibly at intricate regions such as the cat's face and patterned rug textures, where closely spaced edges collapse into indistinct blobs. L_2 loss recovers these fine details more reliably, but its quadratic penalty causes the optimizer to hedge uncertain stroke placements toward grey intermediate values, producing characteristic halo artefacts around edges. The patch loss mitigates both failure modes by combining fine-scale $\mathcal{L}_{fine}$ and coarse-scale $\mathcal{L}_{coarse}$ objectives over a uniform spatial grid, preserving the structural coherence of L_1 while retaining the detail sensitivity of L_2 without the associated blurring.